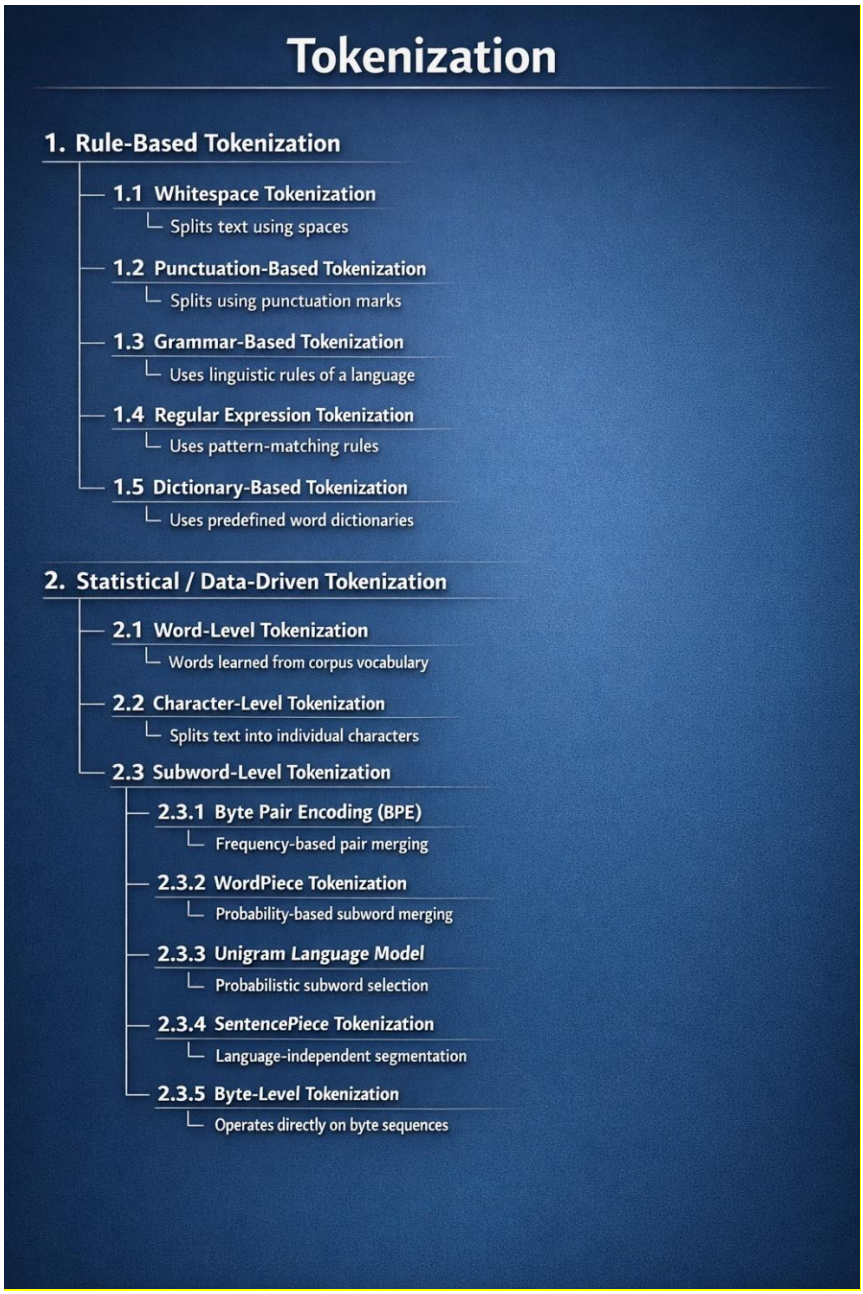
TOKENIZATION

Tokenization is the process of breaking raw text into smaller meaningful units called tokens, so that computers can process, analyze, and understand human language.

TYPES OF TOKENIZATION TECHNIQUES -



RULE BASED TOKENIZATION TECHNIQUES

WHITESPACE TOKENIZATION

Whitespace tokenization is a basic text segmentation technique where tokens are created by splitting text wherever whitespace characters such as spaces, tabs, or line breaks occur.

WORKING –

The tokenizer scans the text sequentially and splits whenever it encounters whitespace characters such as space ( ), tab (\t), or newline (\n). Each continuous sequence of non-whitespace characters becomes a token.

PUNCTUATION BASED TOKENIZATION

Punctuation-based tokenization is a rule-driven technique that segments text by treating punctuation marks (such as commas, periods, question marks, and exclamation marks) as independent token boundaries, separating them from surrounding words.

WORKING –

The tokenizer first separates text into segments and then scans for punctuation symbols. When punctuation is encountered, it splits the symbol into a separate token while keeping adjacent word characters grouped together.

REGEX TOKENIZATION

Regex tokenization is a flexible rule-based text segmentation method that uses predefined pattern-matching expressions to identify token boundaries or extract specific text segments as tokens.

WORKING –

The tokenizer applies predefined regular expression patterns to either split text at matches or extract sequences that match desired token patterns. For example, patterns can be written to capture only words, only numbers, or separate punctuation from words.

STATISTICAL TOKENIZATION

WORD - LEVEL TOKENIZATION

Word-level tokenization is a text segmentation technique in which a sentence is divided into individual words, with each word treated as a meaningful and independent token representing a linguistic unit.

CHARACTER – LEVEL TOKENIZATION

Character-level tokenization is a fine-grained text segmentation method where text is divided into individual characters, treating each letter, digit, punctuation mark, and symbol as a separate token.

BPE

Byte Pair Encoding (BPE) is a subword tokenization algorithm that builds tokens by merging the most frequent pair of adjacent symbols (characters or character sequences) in a text corpus.

Instead of storing entire words, BPE learns frequent subword patterns, allowing rare or new words to be formed from smaller known pieces.

WORKING STEPS -

Step 1: Start with characters as tokens.
Step 2: Count frequency of adjacent symbol pairs.
Step 3: Merge the most frequent pair into a new token.
Step 4: Repeat merging until vocabulary size limit is reached.
Step 5: Tokenize new text using the learned merges.

WORDPIECE

WordPiece is a subword tokenization algorithm that breaks words into smaller meaningful units based on probabilistic scoring.

Instead of merging character pairs purely by frequency (like BPE), WordPiece merges subwords that maximize the likelihood of the training data under a language model.

WORKING –

Step 1: Initialize vocabulary with characters and frequent substrings.
Step 2: Evaluate possible merges using likelihood scores.

Step 3: Select merges that maximize corpus probability.
Step 4: Build vocabulary of probable subword units.
Step 5: Tokenize words using longest-match-first strategy.

WordPiece Working

Training Corpus:
playing, played, player, plays

Step 1 — Initialize with Characters
Split each word into characters:
p l a y i n g
p l a y e d
p l a y e r
p l a y s

Initial vocabulary:
{p, l, a, y, i, n, g, e, d, r, s}

Step 2 — Count Adjacent Pair Frequencies
Common pairs found frequently:
p+l, l+a, a+y appear in all words.

Step 3 — Compute Pair Scores
WordPiece selects merges using probability score:

$$score = \frac{\text{frequency of pair}}{\text{frequency of first token} * \text{frequency of second token}}$$

Pairs that strongly occur together get higher scores.

Step 4 — Merge Highest-Scoring Pair
Suppose "p + l" is selected → merge to form "pl"

Words become:
pl a y i n g
pl a y e d
pl a y e r
pl a y s

Step 5 — Repeat Merge Process
Next high-score merges:

pl + a → pla
pla + y → play

Words become:
play i n g
play e d
play e r
play s

Step 6 — Learn Useful Suffix Subwords

Common endings merged with continuation marker:

i n g → ##ing
e d → ##ed
e r → ##er
s → ##s

Final learned vocabulary includes:
{play, ##ing, ##ed, ##er, ##s}

⚙️ PHASE 2 — Tokenization Using Learned Vocabulary

WordPiece uses Longest-Match-First strategy.

Example 1: "playing"

Longest match from start → play
Remaining → ing → matches ##ing
Tokens: ["play", "##ing"]

Example 2: "player"

Longest match → play
Remaining → er → matches ##er
Tokens: ["play", "##er"]

Example 3: "plays"

Longest match → play
Remaining → s → matches ##s
Tokens: ["play", "##s"]

Final Understanding

WordPiece learns subwords that maximize corpus probability, then tokenizes words by matching the longest known subword first, marking continuations with ##.

SENTENCE PIECE TOKENIZATION

Sentence tokenization (also called *sentence segmentation*) is the process of breaking a large block of text into individual sentences, where each sentence becomes a token.

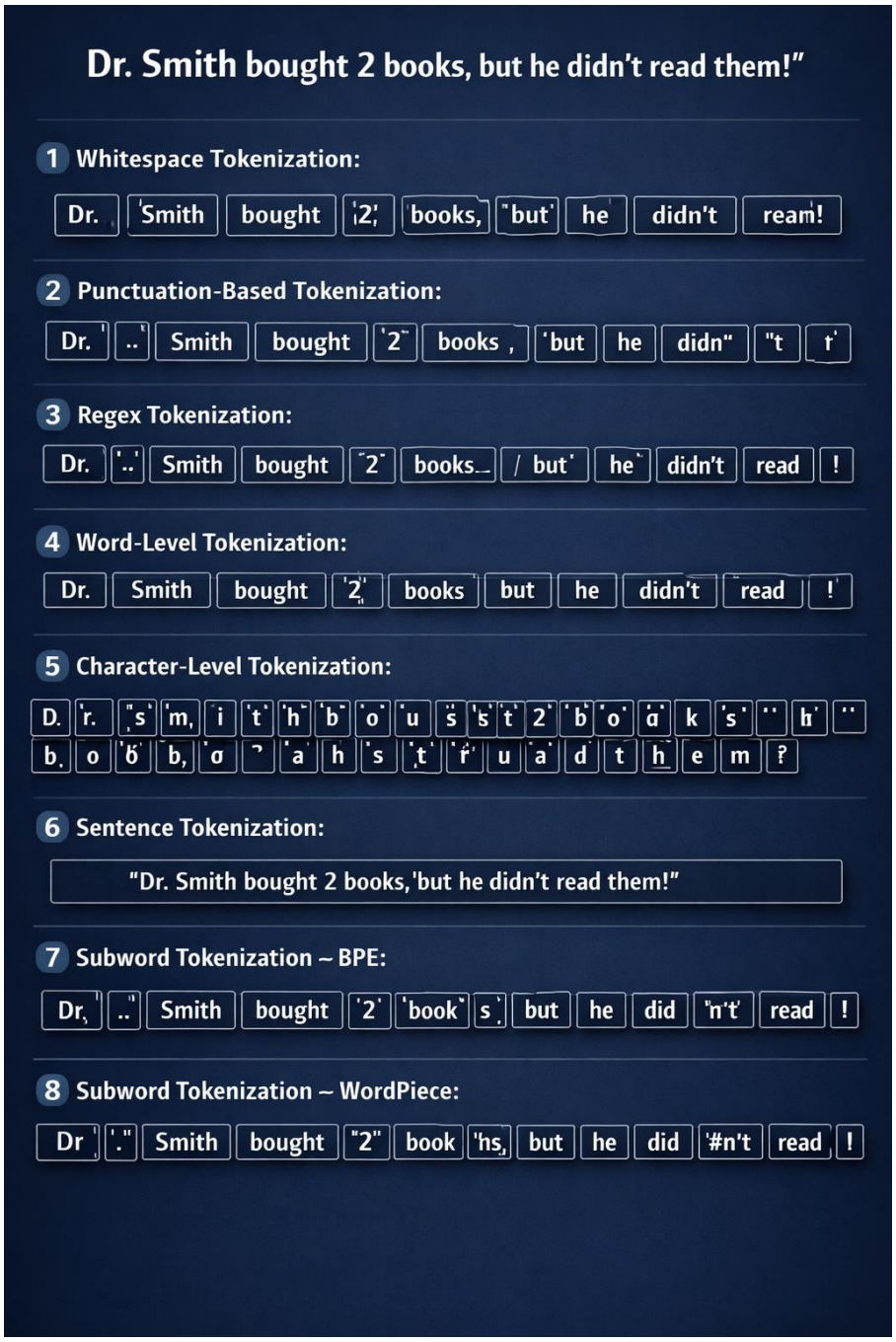
Instead of splitting text into words or characters, this technique focuses on identifying sentence boundaries.

WORKING STEPS ARE –

Step 1: Treat entire text as raw sequence (including spaces).
Step 2: Choose training algorithm (BPE or Unigram).

Step 3: Learn subword vocabulary from corpus statistics.
Step 4: Represent spaces using meta-symbols.
Step 5: Tokenize sentences into learned subword units.

EXAMPLE OF ALL –



Tokenization Technique	LLMs / Model Families Using It
Byte Pair Encoding (BPE)	GPT-2, GPT-3, GPT-4, LLaMA, LLaMA 2, GPT-NeoX, Mistral, Grok-1
Byte-Level BPE	RoBERTa, GPT-2, DeepSeek LLM, DeepSeek-Coder
WordPiece Tokenization	BERT, DistilBERT, ALBERT, ELECTRA
SentencePiece	T5, mT5, BLOOM, Gemma, LLaMA 3